MODULE 18 · DOMAIN 6 - TERRAFORM STATE MANAGEMENT

State Commands, and CLI Workspaces

Editing state deliberately, and the word that means two completely different things.

DOMAIN 6 of 8 COURSE WEIGHT 12% MODULE 18 of 27

admin@sthithapragna.com

A CLI workspace is **another state file for the same configuration**. It is not an environment, and it is not an HCP Terraform workspace.

The state commands

- **terraform state** is a family of subcommands for reading and editing state directly.
- They are the **supported interface** to state. Module four said never to read or edit the file yourself.
- **They act immediately.** No plan, no preview, no approval, no record in a pull request.
- Terraform writes a **backup** before any command that modifies state. Keep it until you are sure.
- Module seventeen's blocks are preferable where they apply. These remain necessary for **inspection**.

> ON THE EXAM
>
> Inspection has no declarative equivalent. That is the enduring reason these commands exist alongside the refactoring blocks.

The subcommands

COMMAND	WHAT IT DOES
`state list`	List the resource addresses tracked in state
`state show ADDRESS`	Print one resource's recorded attributes
`state mv SRC DEST`	Rename or move a tracked object. The moved block does this declaratively
`state rm ADDRESS`	Forget an object without destroying it. removed does this declaratively
`state pull`	Output the raw state, for inspection or scripted backup
`state push`	Overwrite state from a file. The most dangerous command here

ON THE EXAM

list and show are read-only and completely safe. Everything below them changes state immediately.

Inspecting state

- **terraform state list** shows every address, and accepts an address filter to narrow it.
- **terraform state show** prints one resource's attributes as recorded, including computed ones.
- This is how you find the exact address for a **-target**, a **moved** block or an **import** block.
- **terraform show** without arguments prints the whole state, and with -json gives machine-readable output.
- **Neither changes anything.** Both are safe to run at any time, including in production.

> ON THE EXAM
>
> Getting an address right matters more than it looks. list and show are how you check one before using it.

state mv and state rm

- **state mv** changes the address an object is tracked under. The object itself is untouched.
- **state rm** makes Terraform forget an object. The object continues to exist.
- **Neither destroys anything**, which surprises people. They edit belief, not reality.
- Both now have declarative equivalents: **moved** and **removed with destroy = false**.
- After a state rm, the configuration must be removed too, or the next plan proposes to create it again.

> ON THE EXAM
>
> state rm followed by leaving the resource block in place is how people accidentally create a duplicate.

pull, push and replace-provider

- **state pull** fetches and outputs the current state. Useful for a scripted backup before risky work.
- **state push** overwrites the remote state with a local file. It bypasses everything Terraform normally checks.
- Terraform checks **lineage and serial** and will refuse an obviously wrong push, which is why **-force** exists.
- **Never use -force on a push** unless you fully understand the state you are overwriting.
- **state replace-provider** rewrites which provider a resource belongs to, for provider renames or forks.

> ON THE EXAM
>
> Lineage and serial from module four exist for this. They are how Terraform notices you are pushing an unrelated state.

Rules for editing state by hand

- **Take a backup first**, with state pull or by copying the file. Terraform's automatic backup is not a plan.
- **Check the address with state list** before using it. A typo in an address is not always an error.
- **Nobody else should be running Terraform** while you do this. Locking protects applies, not your editing.
- **Prefer the declarative block** where one exists, so the change is reviewed and repeatable.
- Afterwards, **run a plan and confirm it is empty**. That is the only real verification.

> ON THE EXAM
>
> An empty plan after state surgery is the proof that you did it correctly. Anything else means you are not finished.

CLI workspaces

- **Separate instances of state data inside the same working directory.** That is HashiCorp's definition.
- **One configuration, one backend, one set of credentials, several states.**
- Every initialised directory starts with a workspace named **default**, which **cannot be deleted**.
- **Only some backends support multiple workspaces.** It is a per-backend capability, like locking.
- A plan in one workspace **does not see resources in another**. They exist, but you must switch to manage them.

> ON THE EXAM
>
> One backend, several states. Nothing about credentials, permissions or configuration differs between workspaces.

The workspace commands

COMMAND	WHAT IT DOES
`workspace list`	Show all workspaces, marking the current one
`workspace show`	Print the current workspace name
`workspace new NAME`	Create a workspace and switch to it
`workspace select NAME`	Switch to an existing workspace
`workspace delete NAME`	Delete a workspace. Not the default, and not the current one

ON THE EXAM

workspace show before apply is a cheap habit. The current workspace is stored locally and is easy to forget.

Where workspace state actually lives

- **Local backend:** in a directory called **terraform.tfstate.d**, one subdirectory per workspace.
- **Remote backend:** in the same backend, with the workspace name appended to the state path.
- **One backend holds all workspaces.** You do not need a bucket per workspace, and should not create one.
- Workspace names must be valid in a **URL path segment without escaping**, because of how they are stored.
- The **current workspace name is stored locally**, in the ignored .terraform directory, so colleagues can differ.

> ON THE EXAM
>
> The current workspace is a local, per-machine setting. Two people in the same repository can be in different workspaces.

Creating a workspace changes nothing by itself

- **A new workspace gives you an empty state, and the identical configuration.**
- So applying in it builds **a second copy of exactly the same infrastructure**, unless something varies.
- **terraform.workspace** is the named value holding the current workspace name.
- You must **reference it** to get different behaviour: in names, in sizes, in a lookup of per-workspace settings.
- **Forgetting this is the classic mistake.** The workspaces are isolated and the infrastructure is identical.

> ON THE EXAM
>
> Workspaces isolate state. They do not vary configuration. Only terraform.workspace, or a variable file, does that.

Making a workspace behave differently

PATTERN	HOW IT WORKS
Name things after the workspace	Interpolate terraform.workspace into resource names
Look up per-workspace settings	A map keyed by workspace name, read with lookup
Scale by workspace	A conditional: one size in one workspace, another elsewhere
Per-workspace variable files	-var-file passed at the command line. Not auto-loaded
Do nothing	An identical second copy of everything

ON THE EXAM

Terraform will not load a tfvars file named after the workspace. Only terraform.tfvars and *.auto.tfvars auto-load.

HashiCorp's warning

- **Workspaces are not appropriate for deployments needing separate credentials and access controls.**
- The reason is structural: one backend, one credential set, one configuration.
- So **production and development share a blast radius**, and a wrong select is an outage.
- The recommended alternative: **separate configurations** matching architectural boundaries.
- Where they do fit: **short-lived, structurally identical copies** of one stack.

> ON THE EXAM
>
> Enormous amounts of community material recommend workspaces for dev, staging and production. HashiCorp's documentation says otherwise.

CLI workspace or HCP Terraform workspace?

CLI WORKSPACE

A state file selector

- Separate state inside one working directory
- One configuration, one backend
- Shares credentials with every other workspace
- Switched with terraform workspace select

HCP TERRAFORM WORKSPACE

A managed environment

- Its own configuration and working directory
- Its own variables and credentials
- Its own run history, state versions and permissions
- The unit of organisation in HCP Terraform

THE TIEBREAKER

HashiCorp states the two are distinctly different. An HCP workspace is closer to a whole root module than to a CLI workspace.

Reading the scenario

THE WORDING SAYS	IT IS POINTING AT
Separate credentials or accounts per environment	Not CLI workspaces. Separate configurations
I made a workspace and got identical infrastructure	terraform.workspace was never referenced
Rename a resource without recreating it	A moved block, or state mv as the older route
Stop managing it, keep the object	removed with destroy = false, or state rm
Its own variables and run history	An HCP Terraform workspace, not a CLI one
Which resources does Terraform track here?	terraform state list

ON THE EXAM

When a question says 'workspace', decide which kind it means before anything else. The answers differ completely.

> A team proposes managing development, staging and production with three CLI workspaces on one configuration. Each environment lives in a separate cloud account with its own credentials, and only two people may apply to production.
>
> They cite several blog posts describing workspaces as the standard way to handle environments.

What should a reviewer advise?

A. Approve. Workspaces are the documented approach for multiple environments

B. Advise separate configurations. Workspaces share one backend and one credential set

C. Approve, provided each workspace uses a different state bucket

D. Advise HCP Terraform, since CLI workspaces cannot hold more than two states

◆ PAUSE AND DECIDE

What should a reviewer advise?

A. Approve, it is documented

The blog posts say so; HashiCorp says the opposite. Their documentation calls workspaces inappropriate for deployments needing separate credentials and access controls.

B. Separate configurations

Correct. Separate accounts, separate credentials and a permission boundary on production are exactly the conditions the warning names.

C. A bucket per workspace

Misunderstands the mechanism. One backend holds all workspaces, keyed by name. You cannot give each its own bucket without separate configurations anyway.

D. Two-state limit

Invents a limit. There is no cap on workspaces, and HCP Terraform is a reasonable answer here for entirely different reasons.

THE SIGNAL

Three details name the warning exactly: separate accounts, separate credentials, and a permission boundary on production.

> An engineer runs terraform workspace new staging, then terraform apply. The apply succeeds. She then discovers she has a second, complete copy of the production infrastructure, with identical resource names where the platform allowed it.
>
> She expected the staging workspace to produce smaller, differently named resources.

Why did this happen?

A The workspace inherited production's state and reapplied it

B A workspace isolates state but does not vary configuration. Nothing referenced terraform.workspace

C She needed to run terraform init after creating the workspace

D Workspaces require a matching .tfvars file named after the workspace

◆ PAUSE AND DECIDE

Why did this happen?

A It inherited state

The opposite. A new workspace starts with empty state, which is precisely why the apply created everything from scratch.

B Nothing referenced the workspace

Correct. The configuration was unchanged, so applying it against an empty state produced a full second copy.

C Run init

Workspaces do not need reinitialising. Init handles backend, providers and modules, none of which changed.

D A matching tfvars file

Invents a rule. Terraform auto-loads terraform.tfvars and .auto.tfvars only, as module eight established. Workspace names mean nothing to file loading.

THE SIGNAL

The apply behaved perfectly. An empty state plus an unchanged configuration means create everything, and that is what happened.

> An engineer needs to rename a resource in a configuration used by six teams across nine workspaces. Renaming it in configuration alone would destroy and recreate the object.
>
> A colleague suggests running terraform state mv in each workspace. Another suggests a moved block.

Which is better here, and why?

A state mv, because it takes effect immediately without waiting for an apply

B A moved block, because it is applied once per workspace automatically and goes through review

C Neither. Renaming always requires destroy and recreate

D state mv, because moved blocks do not work across multiple workspaces

◆ PAUSE AND DECIDE

Which is better here, and why?

A — state mv, immediate

Immediacy is the drawback, not the benefit. It means nine manual runs, no review, and no record of what was done.

B — A moved block

Correct. It ships with the configuration, so every workspace picks it up on its next apply, and the change is reviewed once.

C — Always destructive

False since Terraform 1.1. moved blocks exist specifically so a rename is not destructive.

D — moved does not span workspaces

Invents a limit. A moved block is part of the configuration, and every workspace uses the same configuration.

THE SIGNAL

Nine workspaces is the deciding detail. It turns a manual command into nine chances to make a mistake.

> A team is planning a migration to HCP Terraform. Their current setup uses four CLI workspaces on one configuration. A manager reads that HCP Terraform also has workspaces and concludes the migration will be a straight one-to-one mapping with no other changes.
>
> An engineer is asked whether that is accurate.

What is the correct characterisation?

A. Accurate. The two are the same concept with different names

B. Not quite. An HCP workspace has its own configuration, variables, credentials and permissions

C. Inaccurate. HCP Terraform does not support multiple workspaces per configuration

D. Accurate, provided the CLI workspaces are renamed to match HCP naming rules

◆ PAUSE AND DECIDE

What is the correct characterisation?

A. Same concept

HashiCorp's documentation calls them distinctly different. A CLI workspace selects a state file; an HCP workspace is a managed environment.

B. Its own everything

Correct. An HCP workspace functions as its own working directory, with separate variables, credentials, run history and permissions.

C. HCP has no multiple workspaces

Backwards. Workspaces are the central unit of organisation in HCP Terraform. There is documented guidance on migrating CLI workspaces to it.

D. Just rename them

Naming is a detail. The manager's error is conceptual, and renaming does nothing about separate variables or permissions.

THE SIGNAL

The migration is genuinely possible and documented. What is wrong is 'with no other changes', because HCP workspaces carry far more.

Module 18 recap

- **state list and show are safe.** mv, rm, push and replace-provider change state immediately.
- **state rm does not destroy anything**, and leaving the resource block behind creates a duplicate.
- **Verify state surgery with an empty plan.** That is the only real proof.
- **A CLI workspace is another state file** for one configuration, one backend, one credential set.
- **HashiCorp says workspaces are not appropriate** where separate credentials or access controls are needed.
- **An HCP workspace is a different thing entirely**, with its own configuration, variables and permissions.

> ON THE EXAM
>
> Creating a workspace changes nothing on its own. Reference terraform.workspace, or you get an identical second copy.